Plano de implantação e migração — Rota Vital

História: PI3-114 (W14 — RSD, Unidade 2) · **Subtarefas:** PI3-199, PI3-200, PI3-201, PI3-202 **Responsável:** Lucas Ferreira Torres de Albuquerque · **Data:** 06/11/2026 **Base:** deploy (PI3-21) · diferenças de topologia (W06) · cenários de SO U2 · benchmarking (PI3-113)

> **Projeto acadêmico.** Todos os dados são sintéticos (LGPD). A telemetria de temperatura e GPS é **simulada**: não existe sensor, veículo nem hardware real. A compatibilidade ABO/Rh e a avaliação de cadeia fria são didáticas. Este plano não trata o sistema como pronto para operação hospitalar real.

---

1. Objetivo e escopo

O plano leva o ambiente de comunicação do Rota Vital do **estado atual** para o **estado-alvo** sem afetar a outra aplicação que divide a VM hoje. O ambiente de comunicação é formado por:

- a API, atrás do Nginx com TLS;
- o domínio;
- o pipeline de deploy;
- o canal de telemetria simulada.

	Estado atual (implantado)	Estado-alvo (este plano)
Servidor	VM `flux` (`us-west1-a`), **978 MB de RAM**, compartilhada com a aplicação Flux	VM **dedicada** ao Rota Vital, 2 vCPU / 4 GiB / 40 GiB, mesma região
Entrada	Nginx + Certbot (TLS), roteando por domínio entre Flux e Rota Vital	Nginx + Certbot, só para o Rota Vital
Aplicação	Spring Boot 21, **uma** instância, systemd, `127.0.0.1:8081`, `-Xmx256m`, SerialGC	Mesma aplicação e mesmo modelo systemd, heap de 1 GiB
Banco	**H2 em memória**: todo reinício apaga os dados	**PostgreSQL 17** local, só em `127.0.0.1:5432`, com backup diário fora da VM
Deploy	GitHub Actions: build → testes → jar → `scp` + `publicar.sh` → health	Mesmo pipeline, com migrations de schema e rollback automático
Domínio	`rsc3-rotavital.duckdns.org`	O mesmo domínio, apontando para o IP **estático** da nova VM
Telemetria	Simulador no GitHub Actions, 1 execução/hora (minuto 7, por 55 min)	Sem mudança, mas passa a ser **persistida**

Por que esse alvo, e não a VPC do diagrama da W04. O diagrama da U1 (W04) desenhava VPC segmentada, balanceador, réplicas, RabbitMQ e Redis. A W06 registrou que nada disso foi implantado e por quê. O dimensionamento de SO U2 recomenda o **cenário moderado** como próximo passo: a necessidade de preservar histórico já justifica a persistência, mesmo com pouca carga. Os demais componentes ficam fora porque nenhuma medição os justifica (ver § 7).

Por que manter Oregon (`us-west1`). O benchmarking da PI3-113 mostrou que ~98% da latência é a distância até Oregon. Mudar para `southamerica-east1` traria o maior ganho de latência, mas o orçamento de SO U2 foi cotado em Oregon. Por isso o plano mantém a região e registra a mudança como decisão separada (ver risco R13).

---

2. Visão geral das fases

```
Fase 0          Fase 1            Fase 2           Fase 3       Fase 4          Fase 5
Código    ──►   Nova VM     ──►   Homologação ──►  Corte   ──►  Estabilização ──► Descomissionamento
(PR)            (em paralelo)     (subdomínio de   (DNS)        (VM antiga em     da VM antiga
                                   teste)                        espera, 7 dias)   ── PONTO SEM RETORNO
◄──────────── VM atual intocada: rollback = não fazer nada ──────────►
```

A estratégia é **implantação paralela com troca de DNS** (*blue-green* com uma máquina de cada cor). A nova VM é montada e testada sem que a atual seja tocada. A troca é uma mudança de IP no DuckDNS, que pode ser desfeita em minutos.

Não há migração de dados. O H2 atual é recriado a cada reinício a partir da carga sintética (`CargaInicial`). Não existe histórico a transferir. O PostgreSQL novo nasce vazio e é populado pela mesma carga inicial, que só roda com o banco vazio (`hemocentros.count() > 0`). Isso simplifica o corte e é o motivo de o rollback antes do corte não perder nada.

---

3. Etapas e pré-requisitos (PI3-199)

Cada fase só começa quando os pré-requisitos estão marcados. O "dono" indica quem pode executar. Etapas na VM exigem acesso ao

projeto GCP, que hoje está com um único integrante (ver risco R11).

Fase 0 — Preparar o código (sem tocar em infraestrutura)

Pré-requisito	Situação hoje
Driver PostgreSQL no `pom.xml`	Já existe (usado pelo `compose.yaml`)
Ambiente local com PostgreSQL	Já existe: `docker compose up` sobe API + PostgreSQL 17 (E1)

#	Etapa	Por quê
0.1	Adicionar **Flyway** e uma migration `V1__schema_inicial.sql` gerada a partir do schema atual	Schema versionado, em vez de gerado pelo Hibernate
0.2	Criar o perfil `prod-pg`, com URL, usuário e senha **só por variável de ambiente** e `spring.jpa.hibernate.ddl-auto=validate`	O perfil `prod` atual usa **`ddl-auto=create`**: num banco persistente, ele **apagaria todas as tabelas a cada reinício** (risco R6)
0.3	Teste de integração contra PostgreSQL (Testcontainers ou o `compose.yaml` no CI)	Hoje o CI testa só com H2; diferenças de SQL apareceriam só em produção
0.4	Estender o `PerfilProducaoTest` ao novo perfil: `ddl-auto` diferente de `create`, nenhuma senha versionada, console do H2 desligado	Mantém as mesmas travas da PI3-21
0.5	Fazer o `publicar.sh` **restaurar o jar anterior sozinho** se o health não responder `UP`	Hoje ele só falha e deixa a versão quebrada no ar (ver § 4.1)
0.6	Paginar `GET /api/v1/bolsas` e o histórico de temperaturas	Com persistência, as listas passam a crescer sem limite. A listagem de bolsas já custa 2 RTT (PI3-113)

Critério de saída: CI verde com PostgreSQL; `prod-pg` sobe numa base vazia, cria o schema pelo Flyway e reinicia sem perder dados. **Dono:** qualquer integrante (só código).

Fase 1 — Provisionar a nova VM, em paralelo

Pré-requisito	Situação hoje
Fase 0 concluída	—
Aprovação do custo do cenário moderado (~**US$ 63/mês**, limite de US$ 72,53 com reserva)	Cotado em SO U2, não aprovado
Acesso de administrador ao projeto GCP	Só um integrante tem (R11)

#	Etapa	Verificação
1.1	Criar a VM `rota-vital` em `us-west1-a`, `e2-custom-2-4096`, disco 40 GiB `pd-balanced`, Debian 12	`gcloud compute instances describe`
1.2	**Reservar IP externo estático**	O IP não muda após reinício. Hoje o IP efêmero é um SPOF marcado no diagrama da U1
1.3	Firewall: 80 e 443 abertas; 22 só para as origens de administração e do GitHub Actions; **5432 e 8081 fechadas**	`gcloud compute firewall-rules list`
1.4	Instalar Nginx, Certbot e PostgreSQL 17 (escutando só em `127.0.0.1`)	`ss -tlnp`: 5432 e 8081 só em loopback
1.5	Criar o banco `rotavital` e um usuário **sem** superusuário; senha em `/etc/rota-vital/ambiente` (permissão 600)	`psql -h 127.0.0.1 -U rotavital`
1.6	Rodar o `infra/provisionar.sh` ajustado: heap `-Xmx1024m`, perfil `prod-pg`, `EnvironmentFile` com as credenciais	`systemctl status rota-vital` (inativo, sem jar)
1.7	Agendar o backup: `pg_dump` diário, criptografado, enviado ao Cloud Storage, guardando 3 cópias	Primeiro dump aparece no bucket
1.8	Agendar **snapshot diário do disco** da VM	Política de snapshot anexada ao disco
1.9	Dar acesso de administrador (IAM / OS Login) a um **segundo integrante**	O segundo integrante entra na VM

Critério de saída: VM no ar, portas internas fechadas, primeiro backup gravado. **Dono:** quem tem acesso ao GCP. **Impacto na produção atual:** nenhum.

Fase 2 — Homologar a nova VM

Pré-requisito	Situação hoje
Fase 1 concluída	—
Subdomínio de teste `rsc3-rotavital-hml.duckdns.org` apontando para o IP novo	—

#	Etapa	Critério de aceite
2.1	Instalar o site do Nginx para o subdomínio de teste, validar com `nginx -t`, emitir o certificado	HTTPS válido no subdomínio
2.2	Publicar o jar à mão (`scp` + `publicar.sh`)	`/actuator/health` responde `UP` local e publicamente; `/h2-console` responde 404
2.3	Testes funcionais dos fluxos principais: cadastro, requisição, alocação FEFO, rota, leitura de temperatura	Respostas iguais às da produção atual
2.4	**Teste de persistência:** criar uma requisição, reiniciar o serviço e confirmar que ela continua lá	Dado sobrevive ao reinício (critério que o H2 não atende)
2.5	**Teste de restauração:** restaurar o último `pg_dump` num banco separado e conferir as contagens	Restauração completa; tempo medido (alvo de RTO: 4 h)
2.6	Rodar o benchmark da PI3-31 contra o subdomínio de teste (3 repetições, espaçadas por 20–30 min)	p95 de leitura ≤ 500 ms, 5xx = 0%, sem `dropped_iterations` (limites de SO U2 § 4)
2.7	Rodar o simulador de telemetria do notebook por 10 min contra o subdomínio de teste	Leituras gravadas e visíveis em `GET /rotas/{id}/temperaturas`

Critério de saída: todos os critérios de aceite atendidos e registrados no Jira. **Dono:** qualquer integrante para 2.3, 2.6 e 2.7; quem tem acesso à VM para as demais.

Fase 3 — Janela de corte

Pré-requisito	Situação hoje
Fase 2 aprovada	—
Janela combinada com a equipe	Fora de apresentação ou entrega. De preferência entre os minutos 02 e 07 de uma hora, quando o simulador de telemetria não está enviando (o workflow roda no minuto 7, por 55 min)
Congelamento da `main` durante a janela	Um push dispara o deploy no meio do corte

#	Etapa	Tempo estimado
3.1	Desativar o workflow "Telemetria simulada" (Actions > Disable workflow)	1 min
3.2	Atualizar o segredo `VM_HOST` do GitHub com o IP novo; gerar uma **nova chave de deploy** e atualizar `VM_CHAVE_SSH`	5 min
3.3	Rodar o pipeline (`workflow_dispatch`) para publicar na VM nova a mesma versão que está em produção	5–8 min
3.4	No DuckDNS, trocar o IP de `rsc3-rotavital` para o IP estático novo	1 min + propagação (TTL de 60 s)
3.5	Instalar o site definitivo no Nginx novo e emitir o certificado de `rsc3-rotavital.duckdns.org`	2–5 min de HTTPS indisponível
3.6	Validar: health `UP` pela URL pública, `/h2-console` 404, `GET /api/v1/hemocentros`, painel `/painel.html`	5 min
3.7	Reativar o workflow de telemetria e confirmar as primeiras leituras gravadas no PostgreSQL	10 min
3.8	Na VM antiga: `sudo systemctl stop rota-vital` e `disable`, **sem remover nada**	1 min

Indisponibilidade prevista: 5 a 15 min, concentrados entre 3.4 e 3.5. **Critério de saída:** passos 3.6 e 3.7 verdes. **Dono:** quem tem acesso ao GCP e ao DuckDNS, acompanhado de um segundo integrante validando de fora.

Fase 4 — Estabilização (7 dias)

Pré-requisito	Situação hoje
Corte concluído	—
VM antiga com o Rota Vital **parado, mas instalado** (pronto para o rollback)	—

#	Acompanhamento	Limite que dispara ação
4.1	`/actuator/health` e uma leitura de telemetria por hora	Health diferente de `UP` por mais de 5 min
4.2	Backup diário presente no bucket	Um dia sem backup
4.3	Memória (`MemAvailable`) e disco	< 512 MiB livres, ou disco > 70%
4.4	Latência e erros, com uma rodada curta de benchmark no D+3	p95 > 500 ms ou 5xx > 1% por 5 min

Fase 5 — Descomissionar o Rota Vital na VM antiga

Pré-requisito	Situação hoje
7 dias sem acionar rollback	—
Um backup restaurado com sucesso **na VM nova**	—
Confirmação da equipe	—

1. Seguir o procedimento "Remover tudo" do deploy.md: parar e remover o serviço, o snippet do Nginx, o sudoers e o usuário `rotavital`.
2. Remover o site `rota-vital` do Nginx da VM antiga, validar com `nginx -t` e aplicar com `reload` (nunca `restart`).
3. Confirmar que o Flux continua respondendo: `curl -o /dev/null -w "%{http_code}" https://rsc3-flux.duckdns.org/`.
4. Revogar a chave de deploy antiga na VM antiga (`~/.ssh/authorized_keys`).

Esta é a última fase reversível. Ver § 4.3.

---

4. Procedimento de rollback (PI3-200)

4.1 Rollback de um deploy comum (vale hoje e depois da migração)

O `publicar.sh` guarda **uma** versão anterior, `rota-vital-anterior.jar`:

```
sudo cp /opt/rota-vital/rota-vital-anterior.jar /opt/rota-vital/rota-vital.jar
sudo systemctl restart rota-vital
curl -s localhost:8081/actuator/health      # esperar {"status":"UP"} (cold start de 20–30 s)
```

Limites do procedimento atual, que a Fase 0 corrige:

Limite hoje	Consequência	Correção
Rollback **manual**	Se o health falha após o deploy, o pipeline fica vermelho, mas a versão quebrada continua no ar até alguém entrar na VM	Etapa 0.5: o `publicar.sh` espera o health e restaura o anterior sozinho
Só **uma** versão guardada	Dois deploys ruins seguidos perdem a última versão boa da VM	Recuperar um jar mais antigo pelos artefatos do Actions (guardados por 7 dias) ou pela imagem `ghcr.io/rsc3-pixel/rota-vital:<sha>`
Exige acesso à VM	Só um integrante consegue executar	Etapa 1.9 (segundo administrador) e 0.5 (rollback automático)

Com PostgreSQL, voltar o jar nem sempre basta. Se a versão nova aplicou uma migration, a versão anterior precisa continuar funcionando com o schema novo. Regra para migrations (*expand/contract*):

1. **Expandir:** primeiro só **adicionar** (coluna nova, opcional; tabela nova).
2. **Contrair:** remover ou renomear apenas num deploy seguinte, depois que nenhuma versão em uso dependa do item antigo.

Ponto sem retorno de um deploy: a aplicação de uma migration **destrutiva** (`DROP`, renomeação ou mudança de tipo). A partir dela, o rollback exige restaurar o backup e perde o que foi gravado desde o último dump (até 24 h). Por isso toda migration destrutiva é precedida de um `pg_dump` manual.

4.2 Rollback da migração, por fase

Fase	Como voltar	Perda de dados	Tempo
0 — código	Reverter o PR	Nenhuma	Minutos
1 — nova VM	Apagar a VM nova e liberar o IP	Nenhuma. Produção não foi tocada	Minutos
2 — homologação	Idem	Nenhuma. Os dados de teste são sintéticos e descartáveis	Minutos
3 — corte (antes de 3.7)	Devolver o IP antigo no DuckDNS, restaurar `VM_HOST` e a chave antiga, e na VM antiga `systemctl enable --now rota-vital`	**Nenhuma.** Ainda não houve gravação relevante no PostgreSQL	5–10 min
3 (depois de 3.7) e 4 — estabilização	Mesmo procedimento	**Perde o que foi gravado no PostgreSQL desde o corte.** A VM antiga volta com o H2 recriado pela carga inicial. Mitigação: `pg_dump` antes de voltar, para reaproveitar na próxima tentativa	10–15 min
5 — descomissionamento	**Não há rollback rápido.** Seria preciso reprovisionar uma VM pelo `provisionar.sh` e pelo deploy.md	Nenhuma no PostgreSQL, mas o ambiente antigo deixa de existir	Horas

4.3 Os dois pontos sem retorno

1. **Sem perda de dados: até o passo 3.7**, quando a telemetria volta e o PostgreSQL passa a acumular histórico. Até ali, voltar para a VM antiga não perde nada, porque o H2 dela nunca guardou histórico.
2. **Sem rollback rápido: a Fase 5**, quando o Rota Vital é removido da VM antiga. Por isso ela só acontece depois de 7 dias estáveis e de uma restauração de backup bem-sucedida.

---

5. Riscos e mitigação (PI3-201)

Os quatro primeiros são os **pontos únicos de falha (SPOF) levantados na U1**, marcados com borda vermelha no diagrama de arquitetura.

Probabilidade e impacto: A = alto, M = médio, B = baixo.

#	Risco	P	I	Mitigação	Risco residual
R1	**VM única (SPOF, U1).** Uma pane na VM derruba Nginx, aplicação e banco juntos. Hoje ela ainda é compartilhada: no benchmark (rep1) a aplicação travou por até 13 s com a rede normal	M	A	VM dedicada (fim da disputa com o Flux); snapshot diário do disco; backup fora da VM; reprovisionamento documentado e versionado; RTO-alvo de 4 h	**Aceito.** Continua uma VM só. Alta disponibilidade exigiria réplica e balanceador, sem requisito que justifique (SO U2)
R2	**Nginx como único ponto de entrada (SPOF, U1).** Uma configuração errada derruba o acesso	B	A	`nginx -t` antes de todo `reload`; nunca `restart`; configuração versionada em `infra/`; verificação pública no pipeline. Na VM nova ele deixa de ser compartilhado	Aceito
R3	**Instância única da aplicação (SPOF, U1).** Todo deploy causa indisponibilidade, somada ao cold start de 20–30 s, com risco de 504	A	M	`Restart=always` no systemd; deploy só pelo pipeline, fora de horário de uso; rollback automático (0.5); aquecimento pós-deploy	**Aceito.** Deploy sem interrupção exige duas instâncias (deploy.md, "O que ficou fora")
R4	**H2 em memória (SPOF de dados, U1).** Todo reinício ou deploy apaga requisições, rotas e telemetria	A (certo)	A	**Eliminado pela migração** para PostgreSQL com backup diário	Substituído por R5
R5	PostgreSQL único na mesma VM: corrupção ou perda do disco	B	A	`pg_dump` diário criptografado no Cloud Storage (3 cópias), snapshot do disco, restauração testada na Fase 2	RPO de até 24 h (meta acadêmica)
R6	**`ddl-auto=create` no perfil de produção.** Num banco persistente, apagaria todas as tabelas a cada reinício	A, se não tratado	A	Flyway + `ddl-auto=validate` (0.1, 0.2); teste no `PerfilProducaoTest` que quebra o build se `create` voltar (0.4)	Eliminado
R7	Migration incompatível com a versão anterior impede o rollback	M	M	Regra *expand/contract*; `pg_dump` antes de migration destrutiva (§ 4.1)	Baixo
R8	**IP efêmero e DNS de terceiro** (DuckDNS gratuito). O IP muda ao reiniciar (marcado no diagrama da U1), e o DuckDNS fora do ar derruba a resolução	M	A	IP estático reservado (1.2). Para o DuckDNS: TTL curto e procedimento documentado; domínio próprio fica como evolução	DuckDNS aceito como dependência externa
R9	Certificado TLS não renovar na VM nova	B	A	Testar `certbot renew --dry-run` na Fase 2; conferir o timer `certbot.timer`	Baixo
R10	Chave de deploy e segredos: vazamento ou uso da chave antiga	B	A	Chave nova dedicada no corte (3.2); revogar a antiga (Fase 5); sudo restrito ao `publicar.sh` (já existe)	Baixo
R11	**Acesso concentrado:** só um integrante administra a VM e o GCP	A	M	Segundo administrador por IAM / OS Login (1.9); rollback automático no pipeline (0.5)	Baixo
R12	Crescimento sem limite das listas e da telemetria (~8.600 leituras/dia) num banco persistente	A	M	Paginação (0.6); política de retenção da telemetria; alerta de disco > 70%	Baixo
R13	Latência dominada pela distância até Oregon (~201 ms por RTT, PI3-113)	A (certo)	B	Mantido para não invalidar a cotação de SO U2; a mudança para `southamerica-east1` fica como decisão separada, com nova cotação	Aceito
R14	Custo de ~US$ 63/mês não aprovado ou recursos esquecidos ligados	M	M	Aprovação antes da Fase 1; alerta de orçamento no GCP; desligar a VM após a avaliação, se a equipe decidir	Baixo
R15	Prejudicar o Flux ao mexer na VM antiga	B	A	A VM antiga só é tocada nas Fases 3.8 e 5, com `nginx -t` + `reload`; canário `curl` no Flux após cada passo	Baixo
R16	Push na `main` ou execução da telemetria durante o corte	M	M	Congelamento da `main`; janela entre os minutos 02 e 07; workflow de telemetria desativado (3.1)	Baixo

6. Coerência com o ambiente implantado (PI3-202)

O plano parte do que está no repositório e no ar, não do diagrama ideal. A tabela mostra cada premissa e onde ela foi conferida.

Premissa do plano	Evidência	Conferido
VM compartilhada `flux` em `us-west1-a`, 978 MB, com o Flux	`docs/deploy.md`; diagrama da U1	Repositório
Aplicação em `127.0.0.1:8081`, perfil `prod`, `-Xmx256m`, SerialGC	`application-prod.properties`; `infra/provisionar.sh`	Repositório
H2 em memória e `ddl-auto=create` em produção	`application-prod.properties`	Repositório
Rollback atual manual, com uma versão guardada	`publicar.sh` (dentro de `provisionar.sh`); `ci.yml`	Repositório
Imagem Docker publicada no GHCR com tag por commit	job `imagem` do `ci.yml`	Repositório
PostgreSQL 17 funcional em container	`compose.yaml`; `docs/evidencias/e1-container.md`	Repositório
Telemetria pelo Actions no minuto 7 de cada hora, por 55 min	`.github/workflows/telemetria-simulada.yml`	Repositório
Latência dominada pela rede; travamento da VM sob carga	PI3-113 (dados brutos de 25/10)	Medição
Cenário moderado: 2 vCPU / 4 GiB, PostgreSQL local, ~US$ 63/mês	SO U2 (dimensionamento e orçamento)	Repositório

Não foi possível conferir na VM (o responsável por este plano não tem acesso ao GCP). Precisa ser confirmado por quem tem acesso antes da Fase 1:

- se o IP atual é efêmero ou estático;
- o tamanho do disco e o tipo de máquina reais;
- se os timeouts do Nginx na VM batem com o `infra/nginx-rota-vital.conf`;
- se o `certbot.timer` está ativo.

Divergências encontradas na documentação, que devem ser corrigidas para o pacote de entrega ficar coerente:

Documento	Diz	Realidade
`evidencias-unidade-1-rsd.md`	VM em `us-central1`; Spring Boot na porta 8080	`us-west1-a`; porta 8081 (deploy.md, provisionar.sh, application-prod.properties)
`deploy.md`, "O que ficou fora"	"HTTPS: falta o certificado"	HTTPS ativo com Certbot (a própria seção 1.5 o emite)
`deploy.md`, seção 1.2	Repositório **privado**	O repositório aceita clone anônimo (público)
`benchmark-rede.md`	Cita o workflow `.github/workflows/benchmark-rede.yml`	O arquivo não existe na `main`. Explica por que as rodadas pelo Actions não foram feitas (PI3-113, § 8)
README e diagrama da U1	"PostgreSQL previsto para a Entrega 02"	A Entrega 02 foi em 21/09 e a produção continua com H2. Este plano é o caminho para isso

---

7. O que este plano não inclui, e por quê

Componente do diagrama da W04	Motivo
VPC segmentada em três sub-redes	Com uma VM só, o isolamento é feito por loopback (8081 e 5432 em `127.0.0.1`) e firewall. A segmentação passa a fazer sentido quando o banco for para outra VM (cenário alto de SO U2)
Balanceador e réplicas da API	Exigem banco compartilhado e teste de consistência da alocação (duas instâncias poderiam reservar a mesma bolsa). Nenhuma medição mostrou saturação até 10 req/s
RabbitMQ e worker de notificação	A ingestão de telemetria é síncrona e leve (~0,1 req/s)
Redis	Nenhum caso de cache definido
Prometheus / Grafana	Hoje o Actuator expõe só o `/health`. Prometheus + Grafana consumiriam mais memória que a aplicação. Em SO U2 a observabilidade do moderado é "tempos por endpoint e métricas internas", sem plataforma dedicada

---

8. Resumo para a apresentação

- **Do quê para o quê:** de uma VM compartilhada com H2, que perde tudo a cada reinício, para uma VM dedicada com PostgreSQL e backup diário. Mesmo domínio, mesmo pipeline.
- **Como:** em paralelo, com troca de DNS. A VM atual só é tocada no corte, e o Flux nunca é afetado.
- **Indisponibilidade:** 5 a 15 min no corte.
- **Rollback:** sem perda até a telemetria voltar a gravar (passo 3.7). Rápido até a remoção do ambiente antigo, que só acontece 7 dias depois do corte.
- **SPOFs da U1:** o H2 é eliminado. VM, Nginx e instância única continuam como **riscos aceitos**, com mitigação e justificativa. O pior caso medido (travamentos de 13 s) vinha da disputa por recursos, que acaba com a VM dedicada.